**Subject :- Cyber Security**

**Practical 2**

**Aim:-** Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

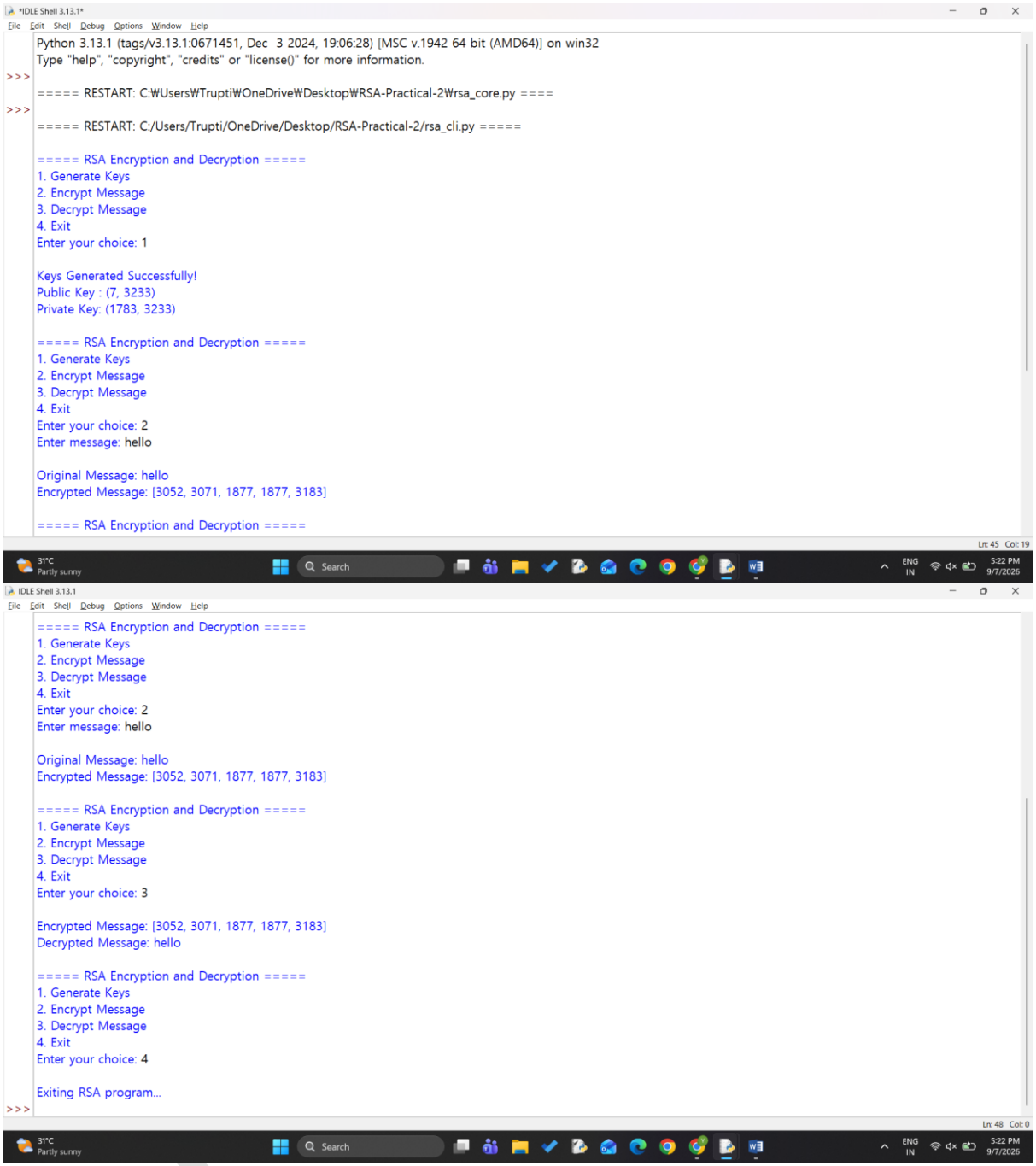
CLI:-

```
from rsa_core import generate_keys, encrypt, decrypt


public_key = None
private_key = None
ciphertext = None



def main():
    global public_key, private_key, ciphertext

    while True:
        print("\n===== RSA Encryption and Decryption =====")
        print("1. Generate Keys")
        print("2. Encrypt Message")
        print("3. Decrypt Message")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            p = 61
            q = 53
```

```
        public_key, private_key = generate_keys(p, q)

        print("\nKeys Generated Successfully!")
        print("Public Key :", public_key)
        print("Private Key:", private_key)

    elif choice == "2":
        if public_key is None:
            print("\nPlease generate keys first.")
            continue

        message = input("Enter message: ")

        try:
            ciphertext = encrypt(message, public_key)

            print("\nOriginal Message:", message)
            print("Encrypted Message:", ciphertext)

        except ValueError as error:
            print("Error:", error)

    elif choice == "3":
        if private_key is None:
            print("\nPlease generate keys first.")
            continue

        if ciphertext is None:
            print("\nPlease encrypt a message first.")
            continue
```

```
            message = decrypt(ciphertext,  private_key)


            print("\nEncrypted  Message:", ciphertext)
            print("Decrypted  Message:", message)


        elif choice == "4":
            print("\nExiting  RSA program...")
            break


        else:
            print("\nInvalid  choice. Please try again.")



if __name__ == "__main__":
    main()
```

**output:-**




**GUI:-**

```
import tkinter as tk

from tkinter import messagebox

from rsa_core import generate_keys, encrypt, decrypt
```

```
class RSAApp:
```

Yash bhagat
T072

```
  def __init__(self, root):
    self.root = root
    self.root.title("RSA Encryption and Decryption")
    self.root.geometry("700x600")

    self.public_key = None
    self.private_key = None
    self.ciphertext = None

    # Title
    title = tk.Label(
      root,
      text="RSA Encryption & Decryption",
      font=("Arial", 20, "bold")
    )
    title.pack(pady=15)

    # Key section
    key_frame = tk.Frame(root)
    key_frame.pack(pady=10)

    tk.Button(
      key_frame,
      text="Generate RSA Keys",
      command=self.generate_rsa_keys,
      width=20
    ).grid(row=0, column=0, padx=10)

    self.public_label = tk.Label(
      key_frame,
      text="Public Key: Not Generated",
```

```
        font=("Arial", 11)
    )
    self.public_label.grid(row=1, column=0, pady=8)

    self.private_label = tk.Label(
        key_frame,
        text="Private Key: Not Generated",
        font=("Arial", 11)
    )
    self.private_label.grid(row=2, column=0, pady=8)

    # Message section
    tk.Label(
        root,
        text="Enter Message:",
        font=("Arial", 12, "bold")
    ).pack(pady=(15, 5))

    self.message_entry = tk.Entry(
        root,
        width=65,
        font=("Arial", 12)
    )
    self.message_entry.pack()

    # Buttons
    button_frame = tk.Frame(root)
    button_frame.pack(pady=20)

    tk.Button(
        button_frame,
```

```
        text="Encrypt",
        command=self.encrypt_message,
        width=15
    ).grid(row=0, column=0, padx=10)

    tk.Button(
        button_frame,
        text="Decrypt",
        command=self.decrypt_message,
        width=15
    ).grid(row=0, column=1, padx=10)

    # Ciphertext
    tk.Label(
        root,
        text="Encrypted Message:",
        font=("Arial", 12, "bold")
    ).pack(pady=(10, 5))

    self.ciphertext_box = tk.Text(
        root,
        height=5,
        width=75
    )
    self.ciphertext_box.pack()

    # Decrypted text
    tk.Label(
        root,
        text="Decrypted Message:",
        font=("Arial", 12, "bold")
```

```
        ).pack(pady=(15, 5))

        self.decrypted_box = tk.Entry(
            root,
            width=65,
            font=("Arial", 12)
        )
        self.decrypted_box.pack()

    def generate_rsa_keys(self):
        # Small primes used for educational demonstration
        p = 61
        q = 53

        self.public_key, self.private_key = generate_keys(p, q)

        self.public_label.config(
            text=f"Public Key: {self.public_key}"
        )

        self.private_label.config(
            text=f"Private Key: {self.private_key}"
        )

        messagebox.showinfo(
            "Success",
            "RSA keys generated successfully!"
        )

    def encrypt_message(self):
        if self.public_key is None:
```

```
            messagebox.showwarning(
                "Warning",
                "Please generate RSA keys first."
            )
            return

        message = self.message_entry.get()

        if not message:
            messagebox.showwarning(
                "Warning",
                "Please enter a message."
            )
            return

        try:
            self.ciphertext = encrypt(
                message,
                self.public_key
            )

            self.ciphertext_box.delete("1.0", tk.END)

            self.ciphertext_box.insert(
                tk.END,
                str(self.ciphertext)
            )

        except ValueError as error:
            messagebox.showerror(
                "Encryption Error",
```

```
            str(error)
        )

    def decrypt_message(self):
        if self.private_key is None:
            messagebox.showwarning(
                "Warning",
                "Please generate RSA keys first."
            )
            return

        if self.ciphertext is None:
            messagebox.showwarning(
                "Warning",
                "Please encrypt a message first."
            )
            return

        decrypted_message = decrypt(
            self.ciphertext,
            self.private_key
        )

        self.decrypted_box.delete(0, tk.END)

        self.decrypted_box.insert(
            0,
            decrypted_message
        )
if __name__ == "__main__":
    root = tk.Tk()
```

```
app = RSAApp(root)
root.mainloop()
```

**OUTPUT:-**